

LCMFreq: Khai thác Tập Phổ Biến

Uno, Kiyomi & Arimura – FIMI 2004

Nhóm 07

CSC14004 – Khai Thác Dữ Liệu và Ứng Dụng

HCMUS · 2025–2026

Định nghĩa cơ bản

- $\mathcal{I}$: tập items; $\mathcal{D}$: transaction database
- **Support**: $\text{sup}(X) = |\{T \in \mathcal{D} : X \subseteq T\}|$
- X **frequent** nếu $\text{sup}(X) \geq \sigma$
- **TID-set**: $\mathcal{T}(X) = \{k : X \subseteq T_k\}$
- Tính chất: $\mathcal{T}(P \cup \{e\}) = \mathcal{T}(P) \cap \mathcal{T}(\{e\})$

Dòng thời gian thuật toán FIM

Năm	Thuật toán	Ý tưởng chính
1994	Apriori	Level-wise candidate gen
1997	ECLAT	Vertical tidset format
2000	FP-Growth	FP-tree, không sinh candidate
2004	LCMFreq	Backtrack + OccDeliver + Hypercube

Hạn chế của các thuật toán trước

- **Apriori**: quét DB đa lượt, sinh nhiều candidate tường minh
- **ECLAT**: bộ nhớ lớn với dense dataset
- **FP-Growth**: FP-tree tốn bộ nhớ, khó cập nhật động

LCMFreq giải quyết bằng 3 kỹ thuật

1. **BackTracking** – DFS trên prefix tree, ít bộ nhớ
2. **OccurrenceDeliver** – đếm support một lần duyệt
3. **HypercubeDecomposition** – batch output không cần đếm lại

1. BackTracking (DFS)

- Xuất phát từ $P = \emptyset$, mở rộng DFS
- Chỉ thêm item $e > \text{tail}(P)$
- Đảm bảo mỗi itemset xuất hiện đúng 1 lần
- Stack chỉ giữ đường đi hiện tại
 $\Rightarrow$ bộ nhớ $O(\|\mathcal{D}\| \cdot \text{depth})$

```
BackTracking(P):  
  output P  
  for e > tail(P):  
    if freq(P ∪ {e}):  
      BackTracking(P ∪ {e})
```

2. OccurrenceDeliver

- Một lần duyệt qua $\mathcal{T}(P)$
- Mỗi $t \in \mathcal{T}(P)$: phân phối vào bucket của từng item $e \in t$
- $\text{Bucket}[e] = \mathcal{T}(P \cup \{e\})$ cho mọi e cùng lúc
- Chi phí: $O\left(\sum_{t \in \mathcal{T}(P)} |t|\right)$

```
for tid in T(P):  
  for e in transactions[tid]:  
    Bucket[e].push(tid)
```

3. HypercubeDecomposition

- $H(P) = \{e : |\text{Bucket}[e]| = |\mathcal{T}(P)|\}$
- $\forall Q \subseteq H(P) : \mathcal{T}(P \cup Q) = \mathcal{T}(P)$
- $S' = S \cup H(P)$; output $2^{|S'|}$ itemsets cùng support
- Không đệ quy cho item trong $H(P)$

```
S' = S ∪ H(P)  
for Q ⊆ S':  
  output P ∪ Q : sup(P)  
for e ∉ S', e > tail(P):  
  if freq: recurse(P ∪ {e}, S')
```

Database ($\sigma = 2$)

TID	Items
T_1	A, B, C, D
T_2	A, B, D
T_3	A, B, C
T_4	B, C
T_5	A, B, C, D
$\mathcal{I} = \{A, B, C, D\}$ $n = 5, \sigma = 2$	

OccurrenceDeliver tại $P = \emptyset$

$\mathcal{T}(\emptyset) = \{T_1, T_2, T_3, T_4, T_5\}, \text{sup}=5$

Bucket	TIDs	sup
[A]	T_1, T_2, T_3, T_5	4
[B]	T_1, T_2, T_3, T_4, T_5	5
[C]	T_1, T_3, T_4, T_5	4
[D]	T_1, T_2, T_5	3

HypercubeDecomposition

$|\text{Bucket}[B]| = 5 = |\mathcal{T}(\emptyset)|$

$\Rightarrow B \in H(\emptyset)$

$S' = \emptyset \cup \{B\} = \{B\}$

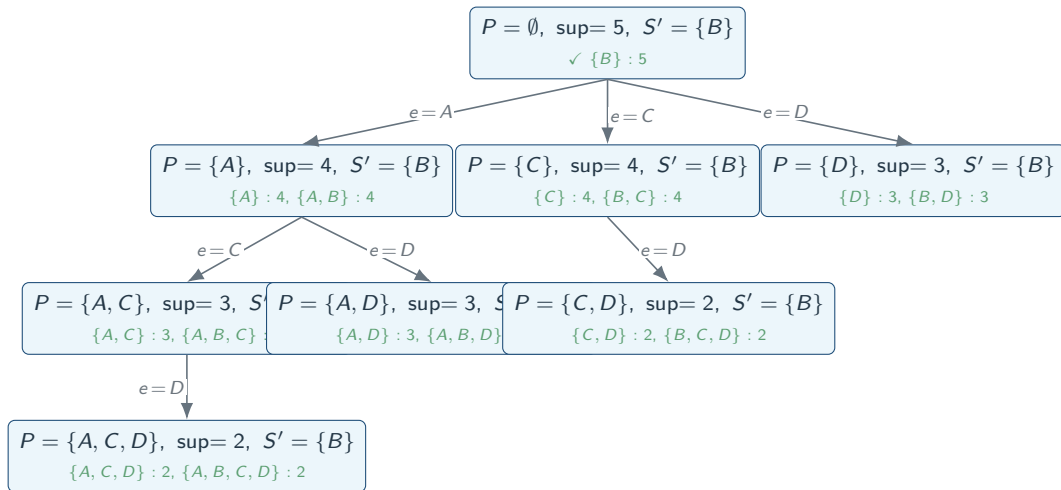
Batch output:

$Q = \emptyset: \{B\} : 5$

Đệ quy với $e \in \{A, C, D\}$

(bỏ $B \in S'$; truyền $S = \{B\}$)

Quan sát: B xuất hiện trong *tất cả* 5 transaction $\Rightarrow B \in H(P)$ tại *mọi* node trong cây DFS. LCMFreq sẽ không bao giờ tạo nhánh đệ quy riêng cho B – thay vào đó B được ghép tự động vào output qua Hypercube.



7 node DFS (thay vì 15 nếu dùng ECLAT). B không bao giờ xuất hiện như nhánh đệ quy – nó được batch-output qua Hypercube tại mọi node.

Chạy tay: Node $P = \{A\}$, $S = \{B\}$

Bước 1 — Tính support

$\mathcal{T}(\{A\}) = \{T_1, T_2, T_3, T_5\}$, sup= 4

Bước 2 — OccurrenceDeliver

Duyệt $\{T_1, T_2, T_3, T_5\}$, điền bucket:

$[B] = \{T_1, T_2, T_3, T_5\}:4$ $[C] = \{T_1, T_3, T_5\}:3$ $[D] = \{T_1, T_2, T_5\}:3$

Bước 3 — Hypercube

$||[B]|| = 4 = \text{sup} \Rightarrow B \in H(\{A\}); S' = \{B\}$

Bước 4 — Batch output

$Q = \emptyset: \{A\} : 4$; $Q = \{B\}: \{A, B\} : 4$

Bước 5 — Dệ quy

$e = C$: sup= $3 \geq 2 \Rightarrow \text{recurse}(P \cup \{C\}, \{B\})$

$e = D$: sup= $3 \geq 2 \Rightarrow \text{recurse}(P \cup \{D\}, \{B\})$

Code: `_hypercube_base!`

```
support_P = length(tids_P) # 1
```

```
local_buckets = Dict{item => Int[] ...} # 2
for tid in tids_P
  for item in transactions[tid]
    haskey(local_buckets, item) &&
    push!(local_buckets[item], tid)
  end end
```

```
H_P = filter(freq_items) do e # 3
  length(local_buckets[e]) == support_P
end
S_prime = sort!(collect(union(S, H_P)))
```

```
for Q in _all_subsets_base(S_prime) # 4
  push!(result, FreqItemset(
    sort!([p[1:plen]; Q]), support_P))
end
```

```
for (j, e) in enumerate(freq_items) # 5
  e <= tail_P && continue
  e in S_prime_set && continue
  _hypercube_base!(p, plen+1,
    local_buckets[e], ..., S_prime)
end
```

Ý tưởng

- Thay `Vector{Int}` (danh sách TID) bằng `BitArray{1}` (bitmap)
- $\mathcal{T}(P \cup \{e\}) = \mathcal{T}(P) \text{ AND } \mathcal{T}(\{e\})$
bằng phép AND theo bit (SIMD, 64 TID/lần)
- $\text{sup}(P) = \text{count}(\text{tidset_P})$
bằng lệnh POPCNT phần cứng

So sánh baseline vs optimized

Phép toán	Baseline	Optimized
Giao tidset	OccDeliver loop	.& SIMD AND
Đếm support	<code>length(v)</code>	<code>count(bits)</code> POPCNT
Lưu trữ	<code>Vector{Int}</code>	<code>BitArray{1}</code>
Dense data	Nhiều phần tử	Ít bit = compact

Code: _hypercube! (optimized)

```
# Giao tidset qua AND bit:
tidset_Pe = tidset_P .& tidsets_full[e]

# Đếm support qua POPCNT:
support_Pe = count(tidset_Pe)

# H(P) via AND:
H_P = filter(freq_items) do e
    count(tidset_P .& tidsets_full[e])
    == support_P
end

# Batch output (giống baseline):
for Q in _all_subsets(S_prime)
    push!(result, FreqItemset(...))
end
```

Kết quả thực nghiệm (tóm tắt)

- Match 100% với SPMF trên 9 tập dữ liệu chuẩn
- Nhanh hơn SPMF Java từ $3.5\times$ đến $141\times$
- Tiết kiệm bộ nhớ tới $27\times$ so với baseline

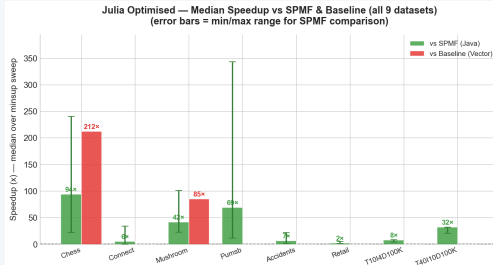
Correctness so với SPMF

Dataset	minsup	Julia	SPMF
Chess	0.75	20 993	20 993
Connect	0.93	7 203	7 203
Mushroom	0.30	2 735	2 735
Pumsb	0.90	2 607	2 607
Accidents	0.80	149	149
Retail	0.02	55	55
T10I4D100K	0.008	494	494
T40I10D100K	0.02	2 293	2 293

Ý nghĩa khi báo cáo

- Output khớp 100% về số lượng itemset.
- Unit test kiểm tra cả support từng itemset trên toy dataset.
- SPMF dùng làm **oracle kiểm chứng**, không sao chép code.

Hiệu năng: Julia Opt vs SPMF



Nhận xét nhanh

Dense dataset như Chess, Mushroom, Pumsb hưởng lợi mạnh từ BitArray: giao tidset bằng AND bit và đếm support bằng POPCNT.

15 frequent itemsets tìm được ($\sigma = 2$)

sup	Frequent itemsets
5	{B}
4	{A}, {A, B}, {C}, {B, C}
3	{A, C}, {A, B, C}, {A, D}, {A, B, D}, {D}, {B, D}
2	{A, C, D}, {A, B, C, D}, {C, D}, {B, C, D}

Điểm nổi bật của LCMFreq

- Chỉ **7 nút DFS** cho ví dụ 15 itemsets (ECLAT xét riêng từng itemset)
- B không bao giờ đệ quy:**
 $B \in H(P)$ tại mọi node $\Rightarrow$ tiết kiệm toàn bộ nhánh B
- OccurrenceDeliver: một lần duyệt $\Rightarrow$ tính support tất cả candidate
- Cài đặt Julia + BitArray:
 AND SIMD + POPCNT phần cứng

Nếu thầy hỏi “paper khác SPMF/code?”

Bản baseline bám sát LCMFreq: Backtracking + OccurrenceDeliver + HypercubeDecomposition. Bản tối ưu giữ khung Hypercube nhưng thay cách đếm support bằng BitArray; đây là tối ưu riêng của nhóm.

Tài liệu tham khảo: Uno T., Kiyomi M., Arimura H. “LCM ver.2: Efficient Mining Algorithms for Frequent/Closed/Maximal Itemsets.” FIMI 2004.